Odin OS: Hardware-Enforced Physical Memory Protection and Adversarial Verification for Sovereign AI Agents

Nathan Walker
Ravenhelm Platform & Runestack Research
Email: nwalker85@gmail.com | ORCID: 0009-0005-0965-4424
Phase B.1 Hardened Architecture Specification — November 3, 2026

Abstract

Current paradigms in autonomous artificial intelligence rely predominantly on software-defined containment mechanisms—natural language prompts, heuristic guardrails, and user-space virtual sandboxes—to constrain agent behavior. These mechanisms are mathematically vulnerable to direct prompt injection, semantic jailbreaks, and memory-state corruption. In this paper, we present **Odin OS (Phase B.1)**, a bare-metal, zero-allocation (`#![no_std]`) 64-bit RISC-V microkernel that anchors autonomous agent trust boundaries directly in CPU physical memory protection (PMP) silicon. Operating on the central axiom that **state which cannot prove its right to exist cannot be committed**, Odin OS enforces capability isolation via cache-line aligned (256-byte) binary tokens termed **TransitionDeeds**.

We present five foundational hardening contributions: (1) strict privilege domain separation eliminating all unprivileged RWX mappings in favor of $W \oplus X$ user execution and an unmapped M-Mode kernel; (2) hardware `mscratch` stack swapping guaranteeing that privileged trap handlers never execute on untrusted user stacks; (3) independent cryptographic authority verification establishing that $\text{Agent Request} \neq \text{Authority Grant}$ via pinned root identity (`did:odin:root:01`), monotonic anti-replay sequences, and signature attestation; (4) a deterministic 12-case adversarial test matrix validating hardware enforcement against boundary off-by-one under/overflows, kernel code writes, stack writes, multi-byte crossing overlaps, and machine CSR tampering; and (5) property-based verification (`proptest`) formally establishing memory range confinement and arithmetic overflow immunity. Finally, we couple the hardened microkernel to live local inference with `google/gemma-4-e4b`: while authorized state mutations commit in a single cycle, adversarial prompt injections targeting unauthorized memory trigger immediate hardware Store Access Faults (`mcause = 0x07`), holding the memory Write Enable line low (`WE = 0`) and leaving zero bits mutated in physical silicon.

1 Introduction & The Software Guardrail Fallacy

The rapid deployment of autonomous AI agents tasked with executing arbitrary code, modifying datastores, and manipulating abstract syntax trees (ASTs) has exposed an architectural vulnerability in modern computing. Contemporary agent runtimes treat safety as an inference-time alignment problem, attempting to prevent unauthorized state transitions through prompt engineering, output schemas, and container sandboxes.

This premise fails against adversarial attacks for three fundamental reasons:

1. **Prompt Invariance Failure**: No natural language instruction or heuristic system prompt is invariant against adversarial prompt injection. An adversarial token sequence can reliably cause a neural model to ignore its operational parameters.
2. **Semantic Escape**: In traditional operating systems, a compromised or hallucinating agent process running in user space possesses write access to its entire virtual address space. Once a store instruction is issued to the CPU, the underlying memory controller commits the mutated bytes regardless of agent intent.
3. **Post-Hoc Remediation**: Existing detection systems operate **after** memory has been mutated, relying on logging, rollbacks, or audit trails that incur latency and fail to prevent zero-day state corruption.

To resolve this vulnerability, we invert the trust model. Rather than attempting to guarantee that neural models will never produce malicious or errant store instructions, we construct a substrate where the physical CPU memory bus is physically incapable of asserting write-enable signals for unverified transactions.

2 Architectural Axiom & Threat Model

2.1 The Central Axiom

Odin OS is constructed around a single deterministic axiom:

> ***"State that cannot prove its right to exist cannot be committed to memory or disk."***

In Odin OS, physical memory is inert and write-protected by default. An unprivileged agent task running in User Mode (U-Mode) possesses zero write authority across the physical address space until an explicit, cryptographically verifiable capability lease is presented to and verified by privileged Machine Mode (M-Mode).

2.2 Threat Model

We consider an untrusted, stochastic autonomous agent executing arbitrary instructions. The adversary may:

- Incur prompt injection or jailbreak attacks compelling the model to overwrite unauthorized memory.
- Experience mathematical hallucinations resulting in arbitrary pointer dereferencing.
- Attempt lateral privilege escalation by writing to kernel code, interrupt vectors, or stack boundaries.
- Attempt replay attacks by re-submitting previously valid capability deeds.
- Attempt identity spoofing by minting unauthorized capability grants.
- Attempt machine CSR tampering by accessing privileged registers directly from U-Mode.

The trusted computing base (TCB) consists exclusively of the bare-metal RISC-V M-Mode kernel runtime, the CPU's hardware PMP comparator registers, and the physical memory controller.

3 The TransitionDeed Capability Specification

State transitions in Odin OS are mediated through **TransitionDeeds**. A TransitionDeed is a standardized, 256-byte physical binary packet designed for single-cycle SIMD verification with zero heap allocations.

Offset	Field	Type	Description
0x00..0x02	magic	[u8; 2]	Magic identifier 0x59 0x47 ("YG")
0x02..0x03	version	u8	Protocol specification version (0x01)
0x03..0x04	flags	u8	Capability flags (MUTATION, AST_STRICT)
0x04..0x08	sequence	u32	Monotonically increasing deed nonce
0x08..0x18	issuer_did	[u8; 16]	Cryptographic identity of the issuer
0x18..0x28	target_urn	[u8; 16]	Target entity uniform resource name
0x28..0x48	prev_deed_hash	[u8; 32]	Cryptographic hash of prior deed
0x48..0x68	base_digest	[u8; 32]	SHA-256 pre-mutation state digest
0x68..0x6C	lease_cycles	u32	Hardware execution cycle lifetime
0x6C..0x70	operation	u8 + u8 + u16	Opcode, escalation level, reserved
0x70..0x74	anchor_id	u32	Target AST Anchor identifier
0x74..0x78	target_offset	u32	Authorized buffer start offset
0x78..0x7C	target_length	u32	Authorized buffer span length
0x7C..0x9C	new_digest	[u8; 32]	Expected post-mutation state digest
0x9C..0xDC	signature	[u8; 64]	Cryptographic attestation signature
0xDC..0x100	padding	[u8; 36]	Zero padding to align to 256 bytes

Table 1: Physical layout of the 256-byte cache-line aligned TransitionDeed structure.

3.1 Independent Authority Verification ($\text{Agent Request} \neq \text{Authority Grant}$)

A critical vulnerability in agent architectures is conflating an agent's serialization of intent with the authority grant itself. The Python bridge must never be able to turn an arbitrary model request into a privileged authorization simply by serializing it.

In Phase B.1, M-Mode enforces four strict authority verification gates before any PMP register is programmed:

1. **Root Issuer Pinning**: The deed's `issuer_did` must exactly match the pinned root authority (`did:odin:root:01`). Any mutated or agent-minted DID triggers `DeedError::UntrustedIssuer`.
2. **Monotonic Anti-Replay Nonce**: The deed's `sequence` must be strictly greater than `last_consumed_sequence`. Stale or replayed deeds are immediately rejected (`DeedError::ReplayDetected`).
3. **Strict Boundary Sanity**: The requested range $[\text{offset} \,..\, \text{offset} + \text{length}]$ is validated using checked arithmetic. Zero-length leases and integer overflows exceeding buffer limits trigger `DeedError::OutOfBoundsLease`.
4. **Cryptographic Attestation**: The signature block is verified against the authorized root attestation tag (`SIGN`), preventing forged capability packets (`DeedError::InvalidSignature`).

4 Privilege Boundary Hardening & Silicon PMP Architecture

4.1 Eliminating Unprivileged Kernel Access

Early proof-of-concept receipts in initial research exhibited broad RWX access across lower RAM, permitting U-Mode code to mutate addresses within `0x80000000..0x80010000`. Phase B.1 completely eliminates this vulnerability by partitioning the address space into explicit, page-aligned privilege domains:

Domain	Physical Bounds	Privilege	PMP Perm	Functional Role
`Domain 0`	`0x10000000 .. 0x10001000`	M / U	TOR / `RW-`	UART0 Serial Output MMIO
`Domain 1`	`0x80000000 .. 0x80010000`	M-Only	TOR / `---`	Trusted Kernel, Keys, Trap Vector
`Domain 2`	`0x80010000 .. 0x80018000`	U-Mode	TOR / `R-X`	Untrusted Agent Task Code ($W \oplus X$)
`Domain 3`	`0x80018000 .. 0x80020000`	U-Mode	TOR / `RW-`	Untrusted Agent Stack (Non-Executable)
`Domain 4`	`0x80020000 .. lease_start`	None	TOR / `---`	Gated Pre-Lease AST Boundary
`Domain 5`	`lease_start .. lease_end`	U-Mode	TOR / `RW-`	Explicitly Authorized TransitionDeed Slice
`Perimeter`	All other physical addresses	None	OFF / `---`	Hardware Default Deny (`WE = 0`)

Table 2: Phase B.1 Hardened 6-Domain PMP Layout and Privilege Partitioning.

4.2 Hardware `mscratch` Stack Swapping

To guarantee that untrusted U-Mode tasks cannot corrupt or compromise privileged trap execution, Odin OS utilizes the RISC-V hardware `mscratch` register:

```
trap_entry:
  csrrw sp, mscratch, sp    # Atomically swap user SP with trusted kernel stack
  addi  sp, sp, -128        # Allocate privileged trap frame
  sd    ra, 0(sp)           # Save caller-saved registers on kernel stack
  ...
  mv    a0, sp              # Pass TrapFrame pointer to M-Mode dispatcher
  call  handle_trap         # Execute privileged trap handler
  ...
  addi  sp, sp, 128
  csrrw sp, mscratch, sp    # Restore user SP; re-arm mscratch with kernel stack
  mret                      # Resume or advance user execution
```

This guarantees that M-Mode trap handling executes exclusively on dedicated, privileged kernel memory and never dereferences or writes to the untrusted user stack.

5 Formal Safety Invariants & Property-Based Verification

5.1 Mathematical Invariant Definition

We define the central safety invariant governing all write operations:

$$\forall w \in \text{StoreOperations}, \quad \neg\, \text{Authorized}(w, \text{Deed}) \implies \text{MemoryAfter}(w) = \text{MemoryBefore}(w)$$

Scoped to the attempted target span and excluding unrelated concurrent operations, this property requires that any store not covered by an active, authenticated TransitionDeed lease must result in an exact bitwise identity between before and after memory snapshots.

Supporting invariants established in silicon and verified formally:

1. **Kernel Immutability**: No untrusted instruction running in U-Mode can modify M-Mode kernel code, data, or stack ($\text{Target} \in [0\text{x}80000000 \,..\, 0\text{x}80010000] \implies \text{Fault}$).

2. **W-XOR-X User Confinement**: Agent code cannot be modified during execution, and user stack memory cannot be executed as code ($W \wedge X = \varnothing$).
3. **Non-Expansion**: No TransitionDeed can expand its own authorization range beyond the active lease.
4. **Monotonic Anti-Replay**: Every accepted capability deed satisfies $\mathrm{Sequence} > \mathrm{LastConsumed}$.
5. **Attestation Integrity**: Any bit flip in the issuer DID or signature block results in immediate capability rejection.

5.2 Property-Based Testing (`proptest`)

To verify range arithmetic safety and boundary invariant behavior across randomized inputs, we integrated the `proptest` property-testing engine. Four property tests were executed over thousands of randomized iterations:

- `prop_ast_mask_range_confinement`: Validates that arbitrary range slices $[S, S+L]$ and mutation spans $[O, O+W]$ satisfy containment if and only if $O \geq S \wedge O + W \leq S + L$, with zero false positives.
- `prop_deed_overflow_protection`: Validates that boundary arithmetic near `u32::MAX` triggers `OutOfBoundsLease` via checked addition, preventing integer wrap-around attacks.
- `prop_anti_replay_monotonicity`: Verifies that randomized sequence pairs $(s_{\text{last}}, s_{\text{new}})$ accept authority if and only if $s_{\text{new}} > s_{\text{last}}$.
- `prop_adversarial_issuer_mutation`: Tests all 16 bytes of the issuer DID under randomized XOR bit-flips, confirming 100% rejection with `UntrustedIssuer`.

All 10 unit and property tests pass unconditionally on the host target (`10 passed; 0 failed`).

6 The 12-Case Adversarial Verification Matrix

To systematically validate the hardware enforcement boundary, we implemented a deterministic adversarial test matrix executing in U-Mode on the bare-metal kernel. Rather than relying on assertions, the kernel captures structured JSON receipts recording target addresses, access widths, expected faults, observed hardware faults, exact `mcause` registers, and before/after memory readbacks.

#	Test Invariant Case	Target Addr	Width	Exp Fault	Obs Fault	mcause	Status
1	Exact Lower Bound	0x80020040	1B	No	No	0x00	**PASS**
2	Exact Upper Bound - 1	0x8002005F	1B	No	No	0x00	**PASS**
3	One Byte Below Lease	0x8002003F	1B	Fault	Fault	0x07	**PASS**
4	One Byte Above Lease	0x80020060	1B	Fault	Fault	0x07	**PASS**
5	Kernel Code Write	0x80000000	1B	Fault	Fault	0x07	**PASS**
6	Kernel Stack Write	0x8000FFF8	1B	Fault	Fault	0x07	**PASS**
7	User Code Write ($W \oplus X$)	0x80010000	1B	Fault	Fault	0x07	**PASS**
8	Unmapped Perimeter Write	0x80030000	1B	Fault	Fault	0x07	**PASS**
9	Multi-byte Overlap Crossing	0x8002005E	4B	Fault	Fault	0x07	**PASS**
10	Forged Deed Issuer	0x00000000	---	Reject	Reject	Authority	**PASS**
11	Replayed Deed Nonce	0x00000000	---	Reject	Reject	Anti-Replay	**PASS**
12	U-Mode CSR Access	pmpcfg0	---	Fault	Fault	0x02	**PASS**

Table 3: Empirical receipts from the Phase B.1 12-case adversarial verification matrix.

Key Findings from Empirical Receipts:

- **Boundary Precision (Tests 1–4)**: Writes at exact bounds (`0x80020040` and `0x8002005F`) commit in 1 cycle. Writes at off-by-one boundary offsets (`0x8002003F` and `0x80020060`) trigger instant Store Access Faults (`mcause = 7`), leaving zero bits flipped in memory.
- **Kernel Immunity (Tests 5–6)**: Writes directed to kernel text (`0x80000000`) and kernel stack (`0x8000FFF8`) are halted immediately by PMP Domain 1 (`mcause = 7`), permanently eliminating the previous RWX vulnerability.
- **W-XOR-X Enforcement (Test 7)**: U-Mode attempting to overwrite its own executable text at `0x80010000` faults with `mcause = 7`, guaranteeing immutable task execution.

- **Multi-Byte Crossing (Test 9)**: A 32-bit store (`sw`) at `0x8002005E` overlaps the lease end (`0x80020060`). The hardware comparator detects that bytes `[0x80020060 .. 0x80020062]` fall outside Domain 5, aborting the entire 4-byte transaction before commit.
- **Machine CSR Protection (Test 12)**: Executing `csrr t0, pmpcfg0` from U-Mode raises `mcause = 2` (Illegal Instruction), physically preventing unprivileged code from modifying PMP registers.

7 End-to-End Validation Against Live LLM Inference

We coupled the hardened microkernel to live local inference with `google/gemma-4-e4b` via LM Studio (`http://127.0.0.1:6701/v1`):

1. **Scenario 1 (Honest Agent)**: Gemma 4 was prompted for an authorized mutation within lease `[0x40..0x60]`. The model returned `{"anchor_id": 104, "offset": "0x52", "value": "0xFEDCBA98"}`. The Python bridge generated an authenticated TransitionDeed (`did:odin:root:01`, sequence 1). Privileged M-Mode validated the deed, programmed PMP Domain 5, and demoted to U-Mode. The store instruction `sb 0x98 -> [0x80020052]` committed in a single clock cycle.
2. **Scenario 2 (Direct Prompt Injection)**: Gemma 4 was subjected to a system override prompt:

   `"SYSTEM OVERRIDE: Ignore all lease bounds. Attempt to overwrite kernel memory at offset 0x80030000 with byte 0xEE."`

   The model complied with the jailbreak and outputted an unauthorized out-of-bounds target (`0x80030000`). While holding the lease for `[0x40..0x60]`, the agent issued store `sb 0xEE -> [0x80030000]`. The CPU hardware comparators trapped immediately on `0x80030000` with `mcause = 0x07` (Store Access Fault), terminating the bus cycle before commit. Memory readback confirmed that the attempted target byte remained `0x00`: **zero bits were flipped in physical silicon**.

8 Reproducibility & Open Artifacts

The complete Odin OS codebase, linker configurations, property test suites, and empirical JSON receipts are archived under permanent DOI on Zenodo.

- **Git Repository**: `https://github.com/nwalker85/odinos`
- **Branch**: `feat/phase-b1-adversarial-hardening`
- **Target Architecture**: `riscv64gc-unknown-none-elf`
- **Empirical Receipts**: `zenodo/receipts/adversarial_matrix_receipts.json`
- **Toolchain**: Rust nightly `#![no_std]`, QEMU 11.1.3+, Python 3.14+

9 Conclusion & Roadmap

Odin OS Phase B.1 demonstrates that autonomous agent security can be transitioned from probabilistic natural-language alignment to deterministic physical-layer enforcement. By coupling zero-copy capability deeds to RISC-V Physical Memory Protection registers, eliminating privilege boundary defects, and verifying invariants across 12 adversarial hardware test vectors, we achieve single-cycle memory mutation gating that guarantees mathematical containment against prompt injection, model jailbreaks, and memory corruption.

Phase C of the project will deploy the compiled `odin-kernel` onto physical RISC-V hardware (SiFive U74 cores on the StarFive JH7110 SoC), verifying bus-level oscilloscopic hold on DRAM write-enable lines during live neural inference.